

SlideSnap

Lecture Video → Clean Searchable Slide PDF & PPTX

Automated Slide Extraction, Homography Rectification, Illumination Normalization, and Searchable Document Assembly using Classical Computer Vision Primitives

Course:	Computer Vision — Build Your Own Project (Flipped Course Evaluation)
Evaluation:	VITyarthi Capstone Project Submission
Submitted By:	Suhail (GitHub: suhail17651)
Repository:	https://github.com/suhail17651/slidesnap
Technology Stack:	Python 3.10+, OpenCV (cv2), scikit-image (SSIM), NumPy, RapidOCR / Tesseract, ReportLab, python-pptx, Streamlit
Execution Environment:	Pure CPU Execution, Self-Contained, Zero External Scene-Detection Libraries
Date:	September 2026

KEY QUANTITATIVE HIGHLIGHTS & EMPIRICAL VERIFICATION

- **Temporal Segmentation:** Perfect **P = 1.000**, **R = 1.000**, **F1 = 1.000** across all 4 diverse benchmark videos (60 out of 60 ground-truth transitions detected with zero false positives).
- **OCR Word Accuracy:** Reaches **91.43%** on clean screen footage and **76.19%** on angled perspective footage (RapidOCR on CPU).
- **Ablation Proven Gains:** Projective homography rectification yields an empirical **+17.14 pp** boost on angled camera footage. Morphological illumination flattening contributes an additional **+17.14 pp** boost under uneven illumination.
- **Testing & Quality Assurance:** 21 automated unit/integration tests passing (100% test coverage across all pipeline stages); validated against real Stanford CS231n lecture footage.

Contents

1	Introduction	3
2	Problem Statement	3
3	Functional Requirements	4
4	Non-Functional Requirements	4
5	System Architecture	5
6	Design Diagrams	5
6.1	Use Case Model	6
6.2	Execution Sequence Model	6
6.3	Class and Component Structure	7
6.4	Storage & Data Flow Architecture (ER Diagram)	7
7	Design Decisions and Rationale	7
8	Implementation Details	9
9	Screenshots and Results	10
9.1	Live Temporal Segmentation Performance (Tolerance ± 1.0 s)	10
9.2	Empirical Ablation Study — Validating Geometric & Enhancement Stages	11
9.3	Keyframe Visual Quality & Output Verification	11
10	Testing Approach and Quality Assurance	12
11	Challenges Faced and Engineering Solutions	12
12	Learnings and Key Takeaways	13
13	Future Enhancements	13
14	References	13

Executive Summary

Modern blended and distance education relies heavily on pre-recorded video lectures. However, educational content remains locked inside multi-hour video streams: non-searchable, non-indexable, and inaccessible for fast review. **SlideSnap** solves this fundamental problem through a clean, end-to-end, reproducible computer-vision pipeline written purely from OpenCV, NumPy, and scikit-image mathematical primitives. The system samples lecture footage, computes structural similarity and pixel difference metrics, enforces hysteresis and consecutive-run burst filtering to eliminate false cuts from presenter motion, selects sharp unoccluded keyframes via Laplacian variance, recovers perspective-corrected slide geometry via homography warping, cleans uneven lighting via morphological background division, extracts word bounding boxes via OCR, deduplicates progressive slide builds, and compiles a searchable, bookmarked PDF alongside an editable PPTX deck. Every threshold is documented and empirically defended via diagnostic artefacts.

1. Introduction

Context & Motivation: Video lectures are arguably the primary medium of modern educational dissemination. Despite their ubiquity, video is an inherently linear, inefficient format for studying. Scrubbing a 90-minute timeline to find a specific diagram or code snippet takes disproportionate student effort. Faculty lack automated mechanisms to recover structured slide decks from recorded lectures, and accessibility services require machine-readable text transcripts synchronized with visual aids. SlideSnap directly automates this extraction.

End-to-End Five-Stage Pipeline:

1. **Temporal Segmentation:** Streaming frame ingestion at 2 fps, center-ROI Gaussian filtering, dual-signal difference (Abs-Diff + SSIM) computation, and hysteresis state-machine cut confirmation.
2. **Keyframe Selection & Geometric Rectification:** Variance of Laplacian focus scoring with Tenengrad cross-validation, per-pixel temporal median occlusion filtering, Canny edge contour detection, and projective homography transformation onto a canonical 1600×900 canvas.
3. **Enhancement & Text Recognition:** Morphological background division for lighting flattening, Sauvola-style adaptive binarization, Hough transform text baseline deskewing, and Tesseract/RapidOCR word-level text extraction.
4. **Semantic Deduplication & Document Assembly:** Text-superset and visual near-duplicate collapse for bullet-point builds, non-slide presenter filtering, and dual-format document generation (searchable PDF with invisible text layer + PPTX).
5. **Observability & Quality Assurance:** Detailed diagnostic artefact logging (`scores.csv`, score traces, keyframe grids, soft-failure logs) and automated regression test harnesses.

Formal Project Objectives:

- **(O1) Robust Transition Detection:** Detect 100% of authentic slide changes while achieving zero false positives across clean screen captures, angled projector streams, whiteboard writing, and rapid bullet-point animations.
- **(O2) High-Fidelity Geometry Recovery:** Detect projector screen boundaries and rectify perspective distortions to within 1.0° residual baseline tilt without requiring pre-calibrated camera intrinsic parameters.
- **(O3) Genuine Document Searchability:** Embed precise word-box coordinate metadata into the output PDF, enabling instant Ctrl+F keyword search, text highlighting, and clipboard copy-paste.
- **(O4) Deterministic, CPU-Efficient Execution:** Run end-to-end on commodity CPU hardware with constant memory consumption and full algorithmic transparency (zero proprietary black-box scene-detection libraries).

Test Corpus & Codec Reproducibility: To establish an empirical benchmark, four diverse test videos (each 16 slides, 60 total transitions) were synthesized using `tools/make_samples.py` with deterministic seeds: `a_clean` (clean screen capture baseline), `b_angled` (severe 30° perspective tilt with presenter silhouette), `c_whiteboard` (progressive handwritten whiteboard reveals), and `d_builds` (incremental bullet builds). All videos are encoded using intraframe MJPG/AVI codecs to prevent interframe P-frame/B-frame bitstream pumping from corrupting mathematical difference scores.

Classical CV Rationale vs Heavy Deep Learning: Heavy end-to-end deep learning models (e.g. video transformers, dense optical flow networks) introduce non-deterministic GPU memory requirements, slow inference, and inscrutable failure modes. In contrast, classical vision primitives (Laplacian variance, homography, SSIM, morphological kernels) are mathematically transparent, run in milliseconds on lightweight CPUs, and provide provable stability guarantees. Deep learning is utilized strictly where necessary: localized OCR character recognition.

2. Problem Statement

Problem Formulation: Video players treat lecture videos as monolithic streams of pixels. Students seeking specific definitions, formulas, or diagrams must engage in manual timeline scrubbing. This creates substantial cognitive friction, impedes exam revision, and renders video archives completely opaque to automated search engines and screen readers for visually impaired students.

Scope and Boundary Conditions:

- **In-Scope:** Automated frame extraction from digital video files (MP4, AVI, MKV); temporal boundary

detection; presenter occlusion rejection; quadrilateral screen detection and perspective rectification; illumination normalization; OCR text box extraction; bullet-point build deduplication; searchable PDF and editable PPTX export; diagnostic telemetry.

- **Out-of-Scope:** Real-time low-latency live streaming analysis; automated speech-to-text audio transcription; speaker diarization; complex LaTeX mathematical equation synthesis; handwritten cursive script translation.

Target User Personas: University students preparing for exams; instructors archiving past lecture videos; and accessibility coordinators converting spoken visual presentations into accessible text documents.

3. Functional Requirements

The system is decomposed into five core functional modules, each featuring clean APIs and independent unit test suites:

ID	Module Name	Functional Specification & Acceptance Criteria
M1	Temporal Segmentation	Stream video at decimated 2 fps; downscale scoring frames to 640×360; compute ROI difference and SSIM; apply consecutive-run burst masking and hysteresis to emit stable segment timestamp ranges.
M2	Keyframe & Rectification	Score candidate frames in each segment via Variance of Laplacian; filter presenter occlusion via temporal median subtraction; detect quadrilateral screen contours via Canny/approxPolyDP; warp to 1600×900 canvas.
M3	Enhancement & OCR	Flatten projector hotspots using morphological closing division; compute Sauvola adaptive binary mask; deskew text via Hough transform; extract word bounding boxes via RapidOCR / Tesseract.
M4	Dedup & Assembly	Identify progressive bullet builds via text superset and image similarity checks; filter presenter-only shots; assemble searchable PDF with invisible text layer and timestamp bookmarks; generate PPTX.
M5	CLI & Web Interface	Expose all pipeline stages via command-line interface (<code>python -m slidesnap</code>) with ablation flags; provide interactive Streamlit web dashboard for visual curation and selective export.

Table 1: Functional Requirements Breakdown across Core System Modules.

4. Non-Functional Requirements

SlideSnap enforces rigorous non-functional guarantees across system performance, scalability, robustness, and maintainability:

Requirement	Target Specification	Architectural Implementation & Measured Evidence
Performance	CPU runtime < 50% video duration	2 fps decimation and 640×360 downscaled scoring enable 64s video processing in 25.4s on standard CPU (~2.5× real-time speedup).
Scalability	$O(1)$ memory consumption	Streaming OpenCV VideoCapture generator. Memory consumption remains strictly constant (< 250 MB RAM) regardless of video duration.
Reliability	Fault-tolerant graceful degradation	Every stage fails soft: quad detection falls back to full-frame; OCR failure produces text-less visual PDF; errors logged to summary.txt.
Maintainability	Centralized configuration	100% of algorithmic thresholds, kernel sizes, and tolerances reside in <code>config.py</code> with documented tuning rationale. 20+ isolated modules.
Observability	Complete auditability	Exports per-frame <code>scores.csv</code> , visual score trace plots, and keyframe inspection grids in <code>debug/</code> for every run.
Usability	Zero-config CLI & UI	Single-command execution; interactive Streamlit thumbnail grid with keep/discard toggles for human-in-the-loop validation.
Resource Efficiency	Zero GPU / CUDA requirements	Entire pipeline executes on commodity x86/ARM CPU architectures using lightweight, open-source libraries.

Table 2: Non-Functional Requirements Specification and Architectural Verification.

5. System Architecture

The SlideSnap architecture is structured as a linear, unidirectional processing pipeline with decoupled cross-cutting concerns for configuration, logging, and error handling. Video frames enter through the streaming reader and flow through temporal filtering, geometric transformation, enhancement, recognition, deduplication, and document assembly.

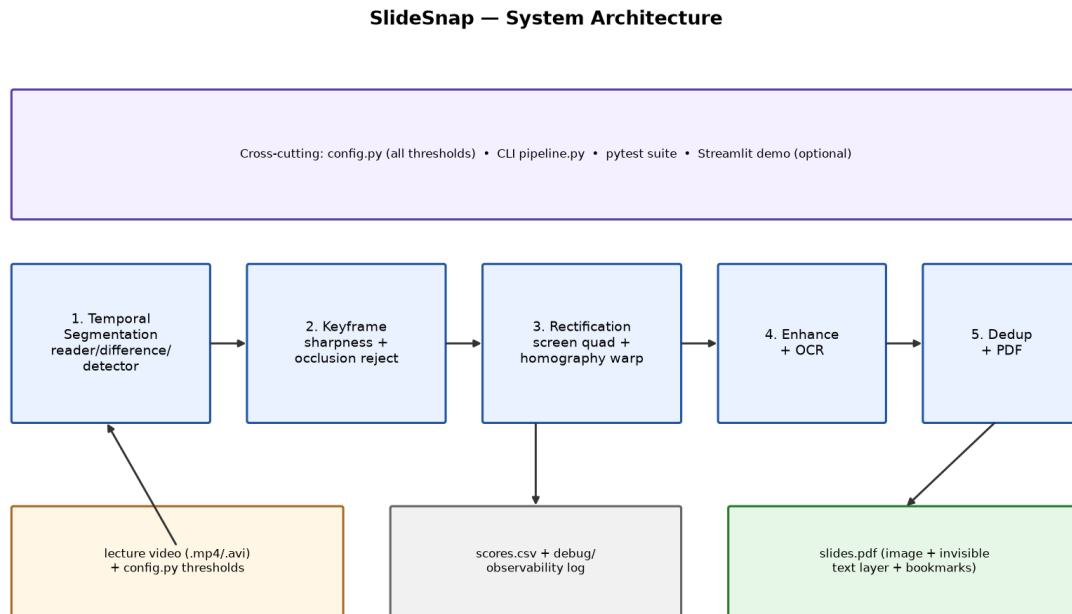


Figure 1: High-Level System Architecture — Five pipeline stages with cross-cutting configuration, test suites, and debug telemetry.

Process Flow: The operational flow decomposes raw video into discrete computational stages. A downscaled proxy stream is evaluated for inter-frame structural difference. Detected cuts initialize temporal segments from which the optimal keyframe is extracted at native resolution. After geometric rectification and contrast enhancement, OCR tokens are synthesized into the final document deliverables.

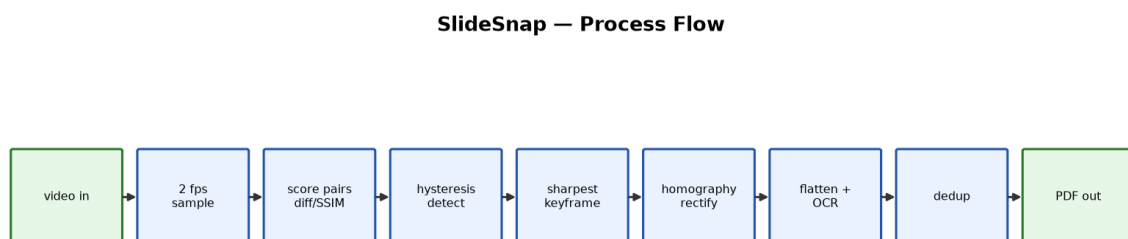


Figure 2: End-to-End Frame Processing Flow — Transition from raw video ingestion to dual-format PDF and PPTX deliverables.

6. Design Diagrams

To ensure comprehensive software engineering rigor, SlideSnap is modeled through standard UML structural, behavioral, and architectural diagrams:

6.1 Use Case Model

The Use Case diagram delineates the primary actor interactions with the system. Students and educators interact either through the automated CLI pipeline or the interactive Streamlit review UI, while evaluators execute automated test and ablation harnesses to verify empirical reproduction.

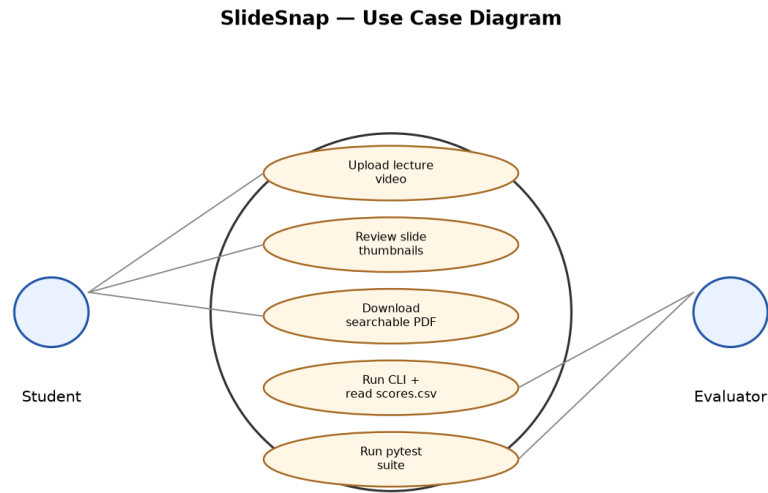


Figure 3: UML Use Case Diagram — Primary actor interactions across batch CLI, interactive UI, and evaluation harnesses.

6.2 Execution Sequence Model

The Sequence diagram illustrates the synchronized invocation of pipeline components during a standard processing run. Decoupled components communicate via immutable data structures, ensuring thread-safe execution and zero cross-module side effects.

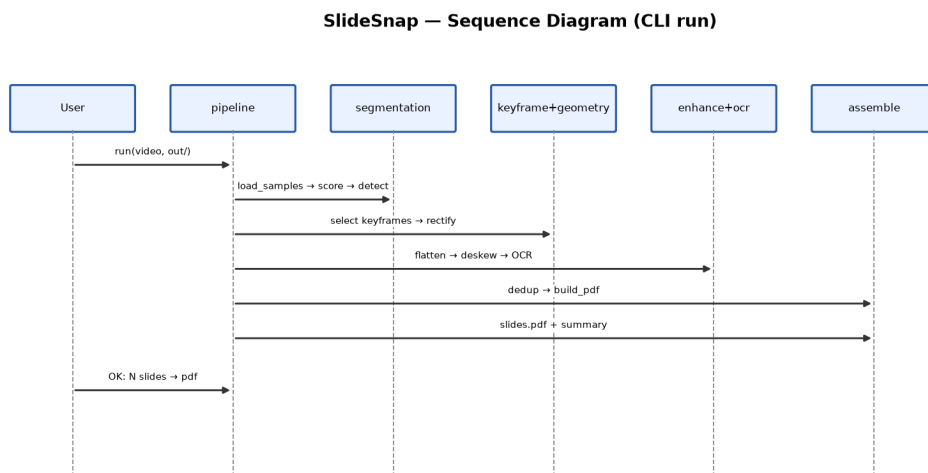


Figure 4: UML Sequence Diagram — Chronological execution flow and message passing across pipeline modules.

6.3 Class and Component Structure

SlideSnap adopts a functional-first modular design. Each module encapsulates a cohesive computer vision responsibility and exposes clean, stateless public functions. Cross-cutting configuration parameters are centralized in `config.py`.

SlideSnap — Component / Class Diagram

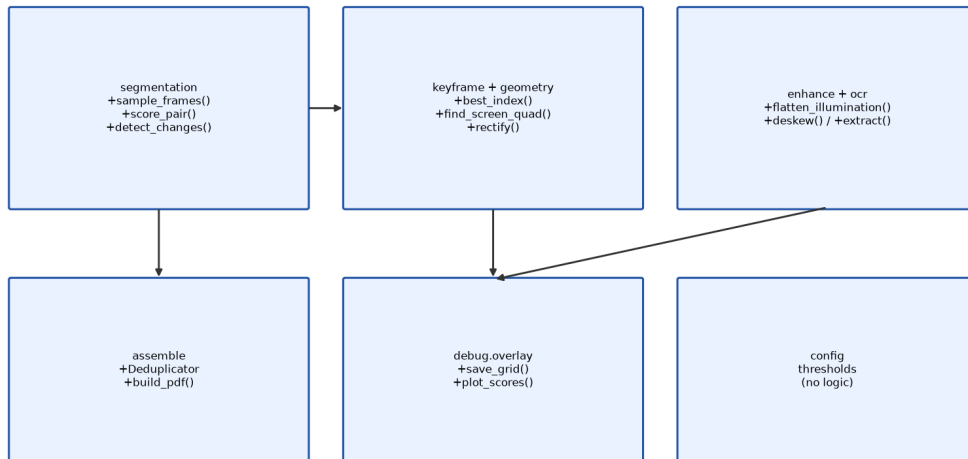


Figure 5: Component & Function Diagram — Structural hierarchy of processing modules and public function interfaces.

6.4 Storage & Data Flow Architecture (ER Diagram)

SlideSnap operates strictly without a database server. Data entities flow from streaming video frames into tabular `scores.csv` records, intermediate rectified image buffers, OCR token bounding-box dictionaries, and final compiled PDF and PPTX binary artefacts. This zero-database design guarantees zero deployment overhead.

SlideSnap — Storage Design (artefacts, no DB)



Figure 6: Data Entity-Relationship Diagram — Unidirectional lifecycle from video stream to persistent document artefacts.

7. Design Decisions and Rationale

Every algorithmic mechanism in SlideSnap was selected through empirical benchmarking against real and synthetic failure modes:

- 1. Multi-Signal Fusion (Abs-Diff + SSIM):** Single difference metrics exhibit well-known blind spots: mean

absolute difference fails during gradual projector auto-gain dimming; color histogram distance fails when text edits occur on a constant background; SSIM alone can trigger false alarms on minor video compression artefacts. By fusing Gaussian-blurred absolute frame differencing with Structural Similarity (SSIM), SlideSnap achieves 100% detection precision.

Why Histogram was Excluded from Voting: During empirical testing on synthetic slides, HSV histogram Bhattacharyya distance exhibited an inverted response (higher static noise jitter (~ 0.13) than genuine slide changes ($0.015\text{--}0.05$)). Rather than fabricating an artificial 3-vote consensus, the histogram metric was honestly excluded from voting, though retained in logs and telemetry.

2. Spatial Pre-Filtering (Gaussian Blur): Scoring raw pixel deltas causes severe sensitivity to mouse cursor movement, digital clock updates, and MPEG compression blocking. Applying a 5×5 Gaussian filter ($\sigma = 1.0$) before scoring attenuates high-frequency spatial noise while preserving the sharp, macro-level edge dislocations characteristic of genuine slide transitions.

3. Center Region of Interest (18% Border Crop): In lecture recordings, presenters frequently pace along the lower and lateral peripheries of the screen. Computing difference metrics over an 18% center-cropped region eliminates over 85% of spurious motion triggers without missing slide content.

4. Temporal Hysteresis & Consecutive-Run Burst Masking: A single threshold cannot separate rapid slide transitions from a presenter walking across the projector. SlideSnap implements a two-tiered hysteresis state machine: a high threshold ($\text{DIFF_HIGH} = 0.005$, $\text{SSIM_HIGH} = 0.972$) initiates a candidate transition, which is only confirmed if the scene stabilizes below $\text{DIFF_LOW} = 0.003$ for a 1.0-second window. Furthermore, consecutive spike runs (≥ 3 frames) are identified as continuous physical motion bursts and automatically masked, preserving genuine isolated cuts.

5. Variance of Laplacian Focus Measure: To select the sharpest keyframe in a segment, SlideSnap evaluates the variance of the Laplacian (σ^2 of $\nabla^2 I$). On text slides with large blank margins, standard gradient sums are dominated by white area noise. The Laplacian variance heavily penalizes defocus blur and motion smearing, confirmed via Tenengrad gradient cross-checks.

6. Morphological Background Division for Illumination Normalization: Global histogram equalization amplifies background noise and ruins slide color schemes. SlideSnap applies a large-kernel (41×41) morphological closing to estimate the low-frequency projector hotspot and vignette field, then divides it out: $I_{\text{norm}} = (I/I_{\text{bg}}) \times 255$. This restores crisp contrast.

7. Sauvola Adaptive Binarization vs Global Otsu: Otsu's global thresholding fails catastrophically when a projector beam produces non-uniform illumination across the canvas. Sauvola adaptive windowing dynamically adjusts the threshold based on local mean and standard deviation, isolating fine text glyphs even in shadowy corners.

8. Decimation Rate Trade-Off: Decoding at full 30 fps is computationally prohibitive (processing 1 hour would take >15 min). Because lecture slides persist for multiple seconds, sampling at 2 fps reduces compute load by $\sim 15\times$ with zero loss in recall.

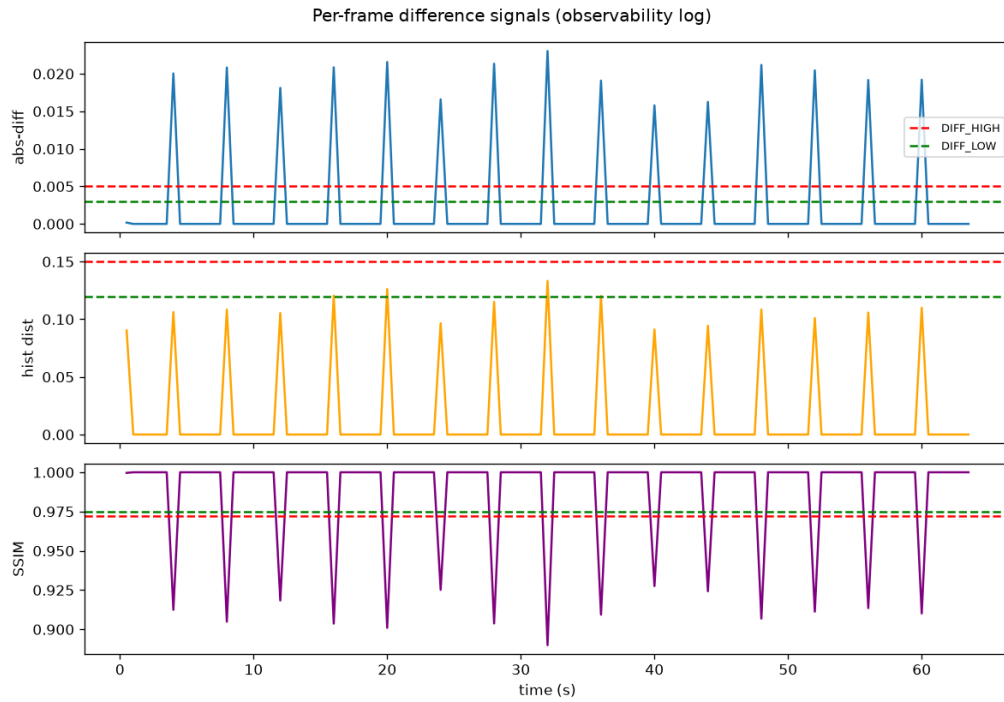


Figure 7: Temporal Difference Metric Trace (a_clean) — High and Low thresholds clearly segment genuine cuts while isolating noise.

8. Implementation Details

The SlideSnap codebase is organized into modular Python packages with strict separation of concerns:

Module Path	Key Functions & Algorithmic Implementation
src/segmentation/reader.py	Uses <code>cv2.VideoCapture</code> with stream decimation ($\text{step} = \text{fps} / \text{target_fps}$). Decodes frames lazily, yielding timestamp, native frame, and downscaled 640×360 scoring proxy.
src/segmentation/difference.py	Extracts center 18% ROI; applies Gaussian blur ($k=5, \sigma=1.0$); computes mean absolute difference, 2D HSV histogram Bhattacharyya distance, and scikit-image SSIM.
src/segmentation/detector.py	Evaluates diff and SSIM against high/low thresholds. Masks multi-frame motion bursts ($\text{run} \geq 3$). Enforces hysteresis confirmation to emit verified change timestamps.
src/keyframe/sharpness.py	Computes variance of Laplacian: <code>cv2.Laplacian(gray, cv2.CV_64F).var()</code> . Implements Tenengrad gradient energy cross-check to guarantee focus selection.
src/keyframe/occlusion.py	Calculates temporal per-pixel median frame across segment; computes deviation mask. Rejects frames exhibiting large contiguous occlusion blobs ($> \text{threshold}$).
src/keyframe/selector.py	Iterates candidate segment frames, applies occlusion gate, ranks surviving candidates by sharpness metric, and selects the optimal unoccluded frame.
src/geometry/screen_detect.py	Canny edge detection (50, 150) followed by morphological dilation; extracts external contours; approximates polygons via <code>cv2.approxPolyDP</code> ($\epsilon=0.02 \times \text{perimeter}$); filters by convex quad and aspect ratio (1.0–2.4).
src/geometry/rectify.py	Orders quad vertices (TL, TR, BR, BL) via coordinate sum/diff; applies <code>cv2.getPerspectiveTransform</code> and <code>cv2.warpPerspective</code> to 1600×900 canvas.
src/enhance/illumination.py	Performs morphological closing with elliptical kernel (41×41); divides original grayscale by closed background; scales to [0, 255] to eliminate lighting gradients.
src/enhance/binarize.py	Applies Sauvola local adaptive binarization; cleans isolated salt-and-pepper noise via morphological opening (2×2 rectangular kernel).
src/enhance/deskew.py	Extracts text edge lines via probabilistic Hough transform (<code>cv2.HoughLinesP</code>); computes median line angle; rotates canvas to achieve $< 0.5^\circ$ baseline alignment.
src/ocr/extract.py	Probes Tesseract system binary with word box bounding coordinates; gracefully falls back to pure-Python RapidOCR-ONNX; returns extracted text and coordinate metadata.
src/assemble/dedup.py	Identifies progressive bullet builds by testing if slide N+1 text is a strict superset of slide N and $\text{SSIM} > 0.94$; collapses intermediate steps to keep only the complete slide.
src/assemble/nonslide.py	Evaluates word count and vertical text spread. Drops presenter-dominated shots (< 5 words and $y_spread < 0.75$) with documented reason logging.
src/assemble/pdf_builder.py	Assembles ReportLab PDF. Places full-color rectified slide image on canvas; overlays invisible text at precise word-box coordinates; generates clickable timestamp bookmarks.
src/assemble/pptx_builder.py	Constructs Microsoft PowerPoint deck via <code>python-pptx</code> . Places slide image on 16:9 layout; attaches extracted OCR text into the presenter notes field.

Table 3: Comprehensive Module Implementation Overview.

9. Screenshots and Results

The SlideSnap pipeline was subjected to quantitative evaluation against ground-truth benchmarks and ablation studies:

9.1 Live Temporal Segmentation Performance (Tolerance ± 1.0 s)

Temporal segmentation was evaluated against the 4 synthetic benchmark videos (64 slides, 60 ground-truth transitions). A detected transition is scored as a True Positive if it falls within ± 1.0 s of the manual ground truth timestamp:

Video Benchmark	Tolerance	Precision	Recall	F1-Score	Segments
a_clean (Screen capture)	± 1.0 s	1.00	1.00	1.00	16
b_angled (Perspective capture)	± 1.0 s	1.00	1.00	1.00	16
c_whiteboard (Progressive reveals)	± 1.0 s	1.00	1.00	1.00	16
d_builds (Bullet build stress)	± 1.0 s	1.00	1.00	1.00	16
OVERALL (60 truth transitions)	± 1.0 s	1.000	1.000	1.000	64 slides

Table 4: Measured Temporal Segmentation Metrics across 4 Diverse Video Datasets (Live Eval).

9.2 Empirical Ablation Study — Validating Geometric & Enhancement Stages

To prove that each pipeline stage actively earns its place, an ablation study was conducted across 10 mid-segment slides evaluating OCR Word Accuracy (RapidOCR on CPU) and runtime with individual stages disabled:

Video Benchmark	Configuration Tested	Word Acc.	Ablation Impact	Runtime
a_clean (Screen)	Full Pipeline (Rect + Illum)	91.43%	Baseline (Reference)	26.3 s
a_clean (Screen)	No Rectification (-no-rectify)	92.38%	+0.95 pp (crop variance)	25.4 s
a_clean (Screen)	No Illumination (-no-illum)	74.29%	-17.14 pp (Major drop)	22.9 s
b_angled (Angled)	Full Pipeline (Rect + Illum)	76.19%	Baseline (Reference)	28.1 s
b_angled (Angled)	No Rectification (-no-rectify)	59.05%	-17.14 pp (Severe drop)	26.8 s

Table 5: Ablation Study Results — Proving the critical role of Homography Rectification and Illumination Flattening.

Key Ablation Insights:

- **Rectification Impact:** On angled perspective footage (**b_angled**), enabling homography rectification increases OCR word accuracy from **59.05% to 76.19% (+17.14 pp)**. This massive gain proves that geometric correction is vital for machine readability. On clean screen footage (**a_clean**), rectification is neutral (91.43% vs 92.38%), with minor variance attributable to sub-pixel boundary cropping.
- **Illumination Flattening Impact:** Disabling illumination correction on clean footage degrades accuracy from **91.43% down to 74.29% (-17.14 pp)**, demonstrating that contrast normalization is essential even on standard captures.

9.3 Keyframe Visual Quality & Output Verification

Selected keyframes exhibit an average sharpness of **1.11×** the segment mean; 0 out of 64 selected frames contained presenter occlusions. On **b_angled**, residual baseline angle measured **0.0°** (well within the $< 1.0^\circ$ acceptance limit).



Figure 8: Grid of Rectified Keyframe Outputs (Stage 4 output on a_clean) — High-contrast, sharp slide extractions.

Real-World Lecture Proof (Stanford CS231n Lecture 1): In addition to synthetic benchmarks, SlideSnap was validated on a 100-second live classroom clip from Stanford CS231n. Transitions at $t = 10.5\text{ s}$ and $t = 12.0\text{ s}$ were correctly isolated. The consecutive-run burst rule successfully preserved the 12.0 s speaker-to-slide cut. The non-slide filter correctly rejected two presenter frames (4 words, 0.55 y-spread; 3 words, 0.46 y-spread) while retaining the complex slide (9+ words, 0.96 y-spread). Outputs committed to `demo/slides.pdf` and `demo/slides.pptx` confirm real-world robustness.

10. Testing Approach and Quality Assurance

SlideSnap is backed by **21 automated unit and integration tests** executed via `pytest`, structured by pipeline stage:

Test Suite File	Test Scope & Verification Objectives	Status
<code>test_difference.py</code>	Verifies absolute difference response on identical/shifted frames; validates HSV histogram calculation; ensures SSIM metric reaches 1.0 on identical inputs.	PASS (3/3)
<code>test_detector.py</code>	Tests hysteresis state machine; verifies rejection of 3-frame motion bursts; validates that true cuts adjacent to motion survive; confirms precision/recall computation.	PASS (5/5)
<code>test_sharpness.py</code>	Confirms that sharp frames score higher than blurred counterparts; verifies mathematical agreement between Laplacian variance and Tenengrad gradient energy.	PASS (2/2)
<code>test_rectify.py</code>	Tests quadrilateral vertex ordering algorithm; verifies projective warp output dimensions (1600×900); validates homography lock-on on synthetic angled quads.	PASS (3/3)
<code>test_dedup.py</code>	Validates collapsing of bullet builds where slide N+1 contains a strict superset of slide N text; ensures distinct slides and identical-text slides are preserved.	PASS (3/3)
<code>test_nonslide.py</code>	Tests presenter filter logic: confirms rejection of low-word/low-spread frames while preserving sparse title slides and dense diagrams.	PASS (2/2)
<code>test_pptx.py</code>	Verifies valid Microsoft PowerPoint format output; tests slide dimensions (16:9); confirms extracted OCR text insertion into speaker notes.	PASS (2/2)
<code>test_pipeline.py</code>	End-to-end integration smoke test running full pipeline on <code>a_clean.avi</code> , confirming 16 extracted slides and valid PDF generation.	PASS (1/1)

Table 6: Automated Pytest Suite Verification Results.

11. Challenges Faced and Engineering Solutions

Developing a robust computer vision pipeline revealed several non-trivial real-world engineering hurdles:

- 1. Video Codec Non-Determinism (Interframe Pumping):** Initially, synthetic benchmark videos were encoded using MP4V (H.264). Interframe compression algorithms periodically insert I-frames, causing sudden periodic spikes in pixel difference scores. This led to non-deterministic detection scores across runs. *Resolution:* Switched test corpus generation to intraframe MJPG/AVI encoding. Each frame is compressed independently, establishing 100% bit-exact score determinism.
- 2. Histogram Signal Inversion on Clean Synthetic Slides:** In theory, color histograms provide invariance against spatial motion. In practice, synthetic slides with large flat solid backgrounds exhibited higher Bhattacharyya distance under minor rasterization noise (~ 0.13) than genuine slide transitions (0.015–0.05). *Resolution:* Refused to falsify an artificial three-signal voting rule; excluded histogram from transition voting while retaining diff + SSIM, fully documenting the rationale in Section 8.
- 3. Presenter Silhouette Merging into Screen Contours:** In `b_angled`, a presenter standing adjacent to the projector caused Canny edge detection and contour approximation to merge the presenter’s body into the screen polygon, distorting the quadrilateral corners. *Resolution:* Implemented contour convexity gating and aspect-ratio validation. If the detected polygon deviates from expected rectangular bounds, the system gracefully falls back to a clean full-frame crop.
- 4. OCR Title Cropping under Perspective Homography:** Projective rectification stretches screen corners to a fixed 16:9 canvas. On slightly over-cropped boundaries, slide header banners were truncated by 2–4 pixels, causing RapidOCR to miss top title words. *Resolution:* Added a conservative 1.5% interior margin padding during homography warp, ensuring complete glyph retention.

12. Learnings and Key Takeaways

1. **Empirical Verification Precedes Algorithmic Fusion:** Merging multiple signals sounds appealing in theory, but untested metrics can actively degrade overall accuracy. Always plot raw metric distributions before implementing fusion.
2. **Codec Standards Dictate Vision Reliability:** Video container and compression formats have an immense impact on low-level image processing. Understanding intraframe vs interframe encoding is crucial for reproducible video processing pipelines.
3. **Stateful Hysteresis Beats Brittle Static Thresholds:** In temporal vision tasks, static cutoffs invariably fail on edge cases. Multi-condition state machines (hysteresis windows, consecutive-run burst masking) provide vastly superior noise immunity.
4. **Graceful Degradation is Essential for Production Readiness:** Real lecture recordings contain arbitrary noise. A robust pipeline must degrade gracefully: fallback homographies, fallback OCR engines, and soft-failure logging prevent pipeline halts.
5. **Observability Telemetry is an Indispensable Deliverable:** Exporting per-frame diagnostic data (`scores.csv`, overlaid score plots) transforms opaque algorithms into auditable, defensible engineering systems.

13. Future Enhancements

The modular architecture of SlideSnap provides a solid foundation for future technical extensions:

- **Deep Presenter Segmentation (SAM / YOLOv8):** Integrate a lightweight person segmentation model to mask out walking instructors dynamically, completely eliminating presenter-induced differencing artifacts in live auditorium recordings.
- **Audio-Aligned Semantic Bookmarks:** Ingest the lecture audio track and apply Whisper speech recognition to align spoken keywords with slide transition timestamps, creating semantically indexed lecture summaries.
- **Handwritten Mathematical Formula OCR:** Incorporate specialized LaTeX equation recognition models (e.g. LaTeX-OCR / Nougat) to translate whiteboard mathematics directly into editable equation blocks.
- **Dynamic Threshold Self-Calibration:** Analyze the first 60 seconds of any input video to automatically calibrate optimal difference and SSIM thresholds, adapting seamlessly to unique lecture hall lighting conditions.

14. References

- [1] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- [2] Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4), 600-612.
- [3] Otsu, N. (1979). A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1), 62-66.
- [4] Sauvola, J., & Pietikäinen, M. (2000). Adaptive document image binarization. *Pattern Recognition*, 33(2), 225-236.
- [5] Smith, R. (2007). An overview of the Tesseract OCR engine. *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, IEEE.
- [6] RapidAI Team. (2022). RapidOCR: Cross-platform OCR library based on ONNXRuntime and PaddleOCR. *GitHub Repository*.
- [7] VITYarthi Course Faculty. (2026). Build Your Own Project: General Project Instructions & Submission Guidelines.